

ensuring that the receiver can detect whether the message has been tampered with during transmission. Although digital signatures can also ensure message integrity, they are computationally more expensive compared to message authentication techniques based on hashing. Before comparing the two, let us look at a simple example of how hashing can be used for message authentication.

Consider the toy hashing algorithm we discussed earlier. Suppose the message is 987, and we know that its hash value is 6. While sending this message, the sender transmits both the original message (987) and its hash value (6). When the receiver gets this combined message, they recompute the hash value of 987 using the same hash algorithm. Since the algorithm is deterministic, the receiver will also obtain a hash value of 6. Because the received hash value and the freshly computed one match, the receiver can be confident that the message has not been altered. However, suppose a malicious user modifies the message so that the receiver gets 986 instead of 987. In that case, the computed hash value will be 5 instead of 6. Since this does not match the hash value sent along with the message, the receiver immediately knows that the message has been tampered with and should not be trusted.

Now, let us have a comparison between digital signatures and message authentication using hashing. Digital signatures and message authentication codes (MACs) both ensure the integrity of a message, but they operate in fundamentally different ways. A digital signature is based on public-key cryptography, where a private-public key pair is used. The sender signs the message using his private key, and anyone with the corresponding public key can verify it. This provides strong sender authentication and non-repudiation, since only the legitimate sender possesses the private key needed to generate the signature. Non-repudiation refers to the security property that ensures a sender cannot deny having created or sent a message later. In other words, once a message is digitally signed, the sender cannot repudiate (or reject) his authorship of that message because the signature uniquely binds the message to his private key. In contrast, message authentication relies on symmetric-key cryptography, where both sender and receiver share a secret key. While MACs also ensure message integrity, they offer only limited sender authentication and do not provide non-repudiation, because either party with the shared key could have generated the authentication code. However, MACs are significantly faster than digital signatures, making them well suited for high-speed or resource-constrained applications.

Before we move on to the actual implementation of these algorithms in SageMath, let us return to another theme we have been exploring throughout this series: what makes SageMath so powerful — often faster than Python and even languages like Julia that are purpose-built for scientific

computing? The key lies in the rich ecosystem of highly optimised mathematical libraries integrated into SageMath. In the previous articles, we examined several of these tools, including FLINT (Fast Library for Number Theory), GMP (GNU Multiple Precision Arithmetic Library), PARI/GP for advanced number-theoretic computations, and the ECM library used for elliptic curve factorisation.

In this article we will explore a very powerful library: the Number Theory Library (NTL). NTL is a C++ library designed for solving number-theoretic problems. Along with a range of high-level mathematical capabilities, NTL supports arbitrary-length integer arithmetic and arbitrary-precision floating-point arithmetic — features that are essential for many cybersecurity-related algorithms. You may recall that FLINT and PARI/GP are also used for number-theoretic applications. So, what distinguishes NTL from these other tools?

First, let us look at the differences between NTL and FLINT. The key distinction lies in their design philosophy and the types of problems each library is optimised to solve. NTL, written in C++, focuses on high-level algebraic and number-theoretic structures. It provides efficient implementations for polynomial arithmetic, lattices, finite fields, modular arithmetic, elliptic curve operations, and related areas that are highly relevant to cybersecurity, making it especially suitable for cryptographic algorithms. FLINT, on the other hand, is a C library designed for extremely fast arithmetic on integers, polynomials, power series, modular forms, and similar objects. Its strength comes from aggressive low-level optimisations and advanced algorithms aimed at achieving raw computational speed, particularly for large-scale arithmetic tasks.

Now, let us examine the differences between NTL and PARI/GP. PARI/GP is a computer algebra system (CAS) designed specifically for number-theoretic computations. It includes both a C library (PARI) and an interactive command line interface (gp) that make it easy to experiment with mathematical ideas. PARI/GP is exceptionally fast for classical number-theoretic tasks, such as integer arithmetic, prime testing, and algebraic number field calculations. In contrast, NTL often outperforms PARI/GP in modular arithmetic, polynomial operations, and lattice-based computations—areas that are especially important in modern cryptography.

Now that we know NTL plays a significant role in cybersecurity applications, let us look at a few examples that demonstrate its computational power. The SageMath program ‘ntl40.sage’, shown below, contains several NTL one-liners that highlight these capabilities.

```
1. print(factor(1111111111111111111111111111111111  
    111111111111111111111111111111))  
2. print(7061709990156159479.is_prime()  
3. print(7061709990156159478.is_prime())
```